

Intuitions sur le versionnement

Intuitions sur le versionnement — Environnements de planning et simulations

Date : 18/04/2026 **Statut :** Décision de conception actée, non encore implémentée **Portée :**
Comment Flux Scheduler gère un plan de production engagé, un travail de planification en cours, et des simulations de faisabilité éphémères

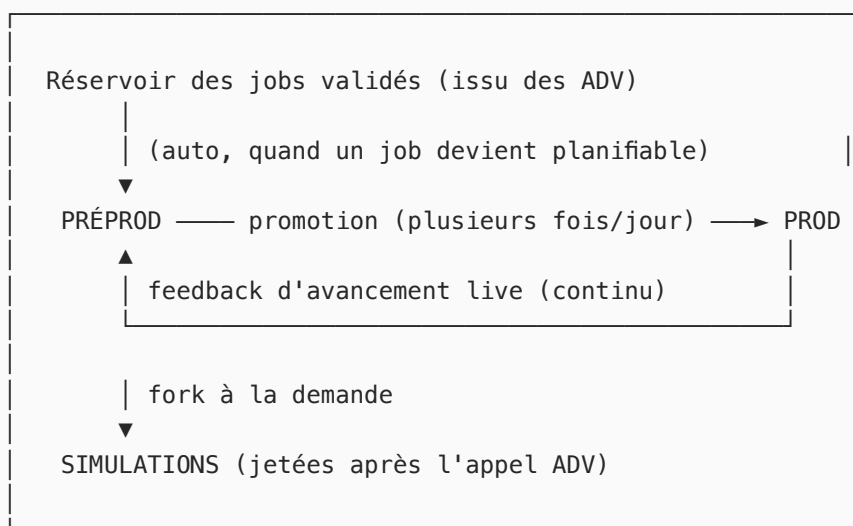
1. La solution retenue

1.1 Trois environnements de planification

Le système sépare le travail de planification en trois environnements aux sémantiques distinctes. Ce ne sont pas des variantes d'un "scénario" générique — ce sont des rôles différents avec des règles différentes.

Environnement	Rôle	Mutabilité	Cardinalité	Durée de vie
Prod	Snapshot gelé de ce que l'atelier exécute	Lecture seule, sauf feedback d'avancement live	1 par société	Remplacée à chaque promotion
Préprod	Workspace de planification — c'est l'app actuelle	Totalement mutable, planif libre	1 par société	Persistante
Simulation	Copie isolée de la préprod utilisée par un ADV pour chiffrer un devis	Mutable dans sa bulle	N (une par appel client)	Courte (15 min typique, TTL)
Archive	Snapshot auto de l'état préprod créé juste avant chaque promotion en prod	Lecture seule	N, croît indéfiniment	Conservée à vie (audit et undo)

1.2 Les trois flux de données



Trois canaux, chacun avec sa direction et sa cadence :

- 1. Réservoir → Préprod** — un job qui devient planifiable (validé par l'ADV, BAT reçu, deadline BAT fixée) est ajouté automatiquement à la préprod avec des valeurs de planification par

défaut. Aucune action utilisateur nécessaire.

2. **Préprod ↔ Prod** — la promotion va vers le bas (manuelle, par le chef d'atelier, plusieurs fois par jour) ; le feedback d'avancement remonte en continu (automatique, ne touche que les champs de completion).
3. **Préprod → Simulations** — un ADV forke une bulle depuis l'état courant de la préprod au moment où il prend un appel client ; la bulle meurt à la fermeture de la session ou à l'expiration du TTL. Jamais promue.

1.3 Comportement d'une simulation

Une simulation est un outil de faisabilité, **pas un espace de staging**. Quand l'ADV répond "oui ça passe" au client :

- La simulation est jetée.
- Le vrai job est créé par le flux JCF normal et entre dans le réservoir par le chemin standard.
- Les décisions de planification prises dans la simulation ne sont **pas** importées dans la préprod.

Ce découplage garantit une seule source de vérité planifiante (la préprod) et évite la classe de bugs où un état hypothétique de simulation s'échappe accidentellement dans la réalité.

1.4 Mécanique de la promotion

Quand le chef d'atelier promeut la préprod en prod :

1. Une archive est créée automatiquement à partir de la prod actuelle (nommée par exemple `Prod 18/04/2026 16:24 (auto-archive avant promotion)`).
2. L'état de planification de la préprod remplace intégralement l'état de planification de la prod.
3. **Exception** : les champs de completion live des tuiles sont préservés depuis la prod actuelle — ils reflètent la réalité d'exécution atelier et ne doivent pas être écrasés par une valeur potentiellement obsolète de la préprod.
4. Fenêtre d'undo de 5 minutes. Passé ce délai, la promotion est finale (mais toujours restaurable depuis l'archive).

Les modifications d'urgence (panne machine, etc.) suivent le même chemin : on édite dans la préprod, on re-promeut. Pas de chemin hot-patch, parce que la promotion est peu coûteuse et fréquente dans ce modèle.

1.5 Principes d'architecture

- **Copies complètes, pas overlays**. Chaque environnement porte une copie complète de toutes les entités planifiantes. Un modèle overlay / delta a été envisagé et rejeté : les use cases réels n'en tirent aucun bénéfice (utilisateur unique par environnement, simulations courtes, promotions fréquentes).

- **scenarioId** sur chaque entité planifiante. Jobs, Tasks, TaskAssignments, Opérateurs, Machines, Groupes, Productivités, état du Schedule — tous portent une clé étrangère **scenarioId**. La prod est une valeur de cet id ; la préprod une autre ; chaque simulation la sienne.
- **Le moteur Rust d'ordonnancement est scenario-aware.** Chaque requête API porte un **scenarioId**. Le moteur lit et écrit uniquement sur les données de ce scénario. Aucune fuite cross-scénario possible.
- **Contexte de scénario dans l'URL côté web.** Le **scenarioId** vit dans l'URL (path ou query param), pas dans **localStorage**. C'est ce qui permet à deux onglets de navigateur de porter des contextes différents indépendamment — prérequis du flux ADV avec plusieurs onglets ouverts.

1.6 Non-goals (explicitement hors-scope)

Pour éviter le scope creep, ces choses ont été envisagées et rejetées :

- **Système multi-scénarios nommés avec couleurs et branches.** Les use cases réels ont besoin de deux environnements fixes + des bulles éphémères, pas d'une forêt de branches.
- **Rôle "approbateur" distinct du planificateur.** Le chef d'atelier est l'approbateur par définition de son rôle. Une entité séparée ajouterait du cérémoniel sans valeur.
- **Co-édition des scénarios par plusieurs planificateurs.** Un seul planificateur par société sur la préprod ; la collaboration multi-utilisateur sur le même environnement est hors-scope.
- **Machinerie de staleness / merge-conflict.** Avec une préprod unique continuellement synchronisée depuis la prod, le problème de staleness classique des systèmes de branches n'apparaît pas.
- **UI générique de gestion de scénarios** (listes, filtres, tags). L'utilisateur ne gère jamais une liste — les environnements sont fixes, les simulations sont éphémères.
- **Modèle de données overlay / delta.** Rejeté après exploration approfondie — cf. section 2.5.

1.7 Principes UX

- **Un workspace par rôle.** Le chef d'atelier vit dans la préprod. Les opérateurs atelier vivent dans la prod (écriture limitée au feedback d'avancement). L'ADV vit dans son onglet de simulation pendant un appel. Aucune ambiguïté sur "dans quel monde suis-je" parce que chaque rôle a une seule réponse.
- **Toggle préprod / prod, pas un switcher.** Le chef d'atelier a dans son UI un toggle à deux états entre la préprod (mutable, son habitat) et la prod (vue read-only de ce qui tourne). Pas de dropdown, pas de switcher multi-scénarios.
- **Nouvelle simulation = nouvel onglet.** L'ADV clique sur "Simuler un nouveau job" (par exemple depuis le flux JCF) ; un nouvel onglet navigateur s'ouvre avec une bulle isolée. Le chrome de l'onglet est visuellement distinct (teinte, bande colorée fine) pour signaler le contexte hors production. Fermer l'onglet met fin à la simulation.

- **La promotion est un rituel distinct.** Flow dédié avec preview du diff, confirmation explicite (checkbox ou retape du nom), création d'archive automatique, toast d'undo post-promotion. La gravité de l'action se reflète dans la friction de l'UI.

1.8 Phases d'implémentation

Un découpage du travail, sans engagement de timeline :

1. **Migration du modèle de données.** Ajout du `scenarioId` sur chaque entité planifiante. Création de la table `scenario` avec les champs type (`prod` | `preprod` | `simulation` | `archive`) et status. Bootstrap du planning existant comme préprod initiale.
2. **API scenario-aware.** Faire en sorte que chaque endpoint PHP accepte et respecte un `scenarioId` . Faire que le moteur Rust accepte un `scenarioId` en entrée et scope toutes ses lectures/écritures en conséquence.
3. **Sync workers (backend).** Deux process background : (a) pousse les jobs nouvellement planifiables du réservoir vers la préprod ; (b) synchronise en continu la completion de la prod vers la préprod.
4. **Flow de promotion.** L'action qui clone l'état préprod dans la prod, préserve la completion live de la prod, crée une archive. Preview de diff, confirmation, fenêtre d'undo.
5. **Vue prod en lecture seule.** Une UI qui rend la planification prod courante, désactive toute mutation sauf le feedback d'avancement. Probablement un toggle dans l'app existante.
6. **Création et destruction de simulation.** Point d'entrée "Simuler un nouveau job" ; flow en nouvel onglet avec une copie complète forkée de la préprod ; auto-nettoyage TTL.
7. **Navigation des archives.** Une vue simple des snapshots historiques de la prod, en lecture seule, utile pour l'audit et la récupération au-delà de la fenêtre d'undo de 5 min.

2. Le chemin de réflexion

Cette section capture le cheminement de pensée. Elle s'adresse à un futur lecteur qui hériterait de cette conception et se demanderait *pourquoi pas X, pourquoi pas Y*. Préserver les branches d'exploration abandonnées est souvent plus utile que la réponse finale seule.

2.1 Point de départ

La question initiale était ouverte : *"Je veux un planning et des settings complets 'de prod' (jobs, opérateurs, machines...) et pouvoir faire autant de branches que je veux pour tester différents plannings, priorités, etc."* L'utilisateur a demandé une discussion avec trois voix spécialistes : UX, planification industrielle, versionnement type git.

Cette formulation suggérerait un système multi-scénarios riche, comparable aux branches git appliquées à la planification.

2.2 Exploration spécialiste initiale

Chaque perspective a apporté :

- **Planification industrielle** a confirmé que les APS (SAP APO, Dassault Quintiq, Siemens Opcenter) ont ce concept depuis longtemps, appelé *planning versions* ou *scénarios*. Il vient avec des pièges connus : les scénarios vieillissent vite, les utilisateurs confondent les environnements, les KPI doivent être scopés par scénario.
- **UX** a insisté sur le modèle mental : les utilisateurs ne doivent jamais oublier dans quel environnement ils sont. Figma branching, Excel Scenario Manager, Google Docs versions ont été cités comme inspirations concrètes.
- **Versionnement logiciel** a clarifié quelles primitives git se traduisent bien (branch, commit, diff, tag) et lesquelles ne se traduisent pas (merge 3-way textuel, rebase linéaire) pour un domaine à graphe contraint comme la planification.

À ce stade, l'hypothèse par défaut était : *un scénario est une branche nommée, persistante, colorée, de l'état planning complet.*

2.3 Six questions de cadrage

Les premières réponses ont resserré la conception :

1. Utilisateurs : à terme plusieurs planificateurs.
2. Durée de vie d'un scénario : quelques heures, "statique" (pas édité en continu).
3. Promotion : oui, un scénario peut devenir la prod.
4. Nombre : typiquement 2-3 en parallèle, pas de plafond.
5. Évolution de la baseline pendant la vie d'un scénario : non applicable (scénarios courts).
6. Scope : tout — jobs, priorités, opérateurs, machines, groupes, productivités.

Ces réponses pointaient vers : **snapshots nommés en copie complète** comme approche de départ, avec évolution possible vers un overlay si l'usage croissait.

2.4 Itérations du playground

Trois itérations du playground HTML ont exploré l'UI :

- **V1** était confus : il incluait un mode comparaison côte-à-côte (lecture erronée de "je voudrais deux onglets" comme "je voudrais une vue splittée") et utilisait un layout fake générique qui ne ressemblait pas au vrai Flux Scheduler.
- **V2** a corrigé les deux : une seule fenêtre à la fois, layout réel de la Flux Toolbar + tabs, pattern bandeau-est-le-switcher où le nom du scénario lui-même est le dropdown cliquable (à la GitHub/Linear/Figma).
- **V3** a exploré une alternative : bande fine colorée en haut + bouton d'action flottant (FAB) en bas à droite. Le mode production est visuellement neutre (pas de bande, FAB gris) ; le mode scénario ajoute la bande, un halo subtil au viewport, et des boutons d'action colorés. Le

principe : **l'ambient awareness par absence** — la normalité est la prod, un scénario est une exception marquée.

2.5 Le détour overlay

Les considérations multi-utilisateurs ont déclenché une exploration architecturale majeure : le modèle **réalité partagée + overlay par scénario**. Inspiré de Kubernetes Kustomize, Docker layers, Figma component instances, vues Airtable.

L'idée : les entités se répartissent sur deux couches.

- **Réalité partagée** — jobs, opérateurs, machines, completions, événements : données qui viennent du monde réel et doivent être visibles instantanément dans tous les scénarios.
- **Overlay par scénario** — priorités, affectations de tasks, configs de groupes, productivités : la configuration que le scénario teste.

Ce modèle aurait résolu élégamment le problème de staleness (les nouveaux jobs se propagent automatiquement dans tous les scénarios) et autorisé des opérations structurelles (exclusions de tasks, ajouts, rewiring de dépendances) via des patches style Kustomize.

Il a finalement été abandonné. Quand les use cases réels ont émergé, aucun ne justifiait cette complexité. Le modèle overlay était une belle réponse à un problème que l'utilisateur n'avait pas.

2.6 Le retour à la réalité

Le point de bascule : l'utilisateur a pris du recul et décrit le workflow réel.

"Un ADV répond au téléphone à un client qui demande 'vous avez la place pour un dossier de 15 000 brochures avant le 24 avril ?' L'ADV crée une copie complète de la prod, y met le job du client au téléphone, triture pour voir si ça passe. Ce planning n'est jamais promu."

"Le chef d'atelier a UNE copie du planning de prod, disons comme une préprod. Quand un job est validé par les ADV, il s'ajoute dans la préprod. Quand la préprod convient au chef, il la pousse en prod, tout s'écrase sauf l'avancement des tuiles."

Ces deux narrations ont fait sauter le frame générique "scénarios". Elles ont révélé :

- Deux patterns distincts aux sémantiques différentes, pas des variantes d'un seul concept.
- Pas de collaboration multi-utilisateur sur un même plan (un seul planificateur par société).
- Pas de conflit de promotion cross-scénario (seule la préprod est promue).
- Pas de souci de staleness (le flux continu gère ça).

Le système de scénarios riches était sur-ingéniéré pour les besoins réels.

2.7 Convergence vers trois environnements

Le modèle final a émergé en nommant chaque use case explicitement :

- Le travail en cours du chef d'atelier → **Préprod** (persistante, une par société).
- L'outil de faisabilité de l'ADV → **Simulation** (éphémère, N par société).
- Le plan engagé pour l'atelier → **Prod** (gelée, lecture seule sauf completion).
- Le filet de sécurité de la promotion → **Archive** (snapshots historiques, lecture seule).

Pas d'overlay, pas de métaphore branche, pas de switcher entre scénarios colorés. Trois environnements distincts avec des règles de sync spécifiques entre eux.

2.8 Validation comme pattern planning-vs-execution

L'intuition finale est venue de l'utilisateur : *"Ce qu'on a aujourd'hui c'est déjà la préprod. La prod est read-only — on peut juste y donner du feedback d'avancement."*

Cette observation s'aligne avec la séparation industrielle classique entre **planification** (mutable, élaborée, débattue) et **exécution** (gelée, engagée, suivie). SAP appelle ça *Planned Order* vs *Process Order* ; les autres APS *Preliminary Plan* vs *Committed Schedule* ; le software dev vs prod. Dans tous les cas, le rituel de *commit/publish/promote/release* sanctifie la frontière entre "libre de modeler" et "en cours d'exécution".

Ce cadrage a fait plus que valider la conception — il a révélé que l'app actuelle est déjà aux deux tiers du chemin. Le comportement préprod existe déjà ; ce qui manque, c'est la frontière prod explicite et le rituel de promotion.

3. Intuitions à conserver

Voici les leçons réutilisables du chemin parcouru, plus larges que cette feature précise :

- **Ne pas créer des abstractions génériques ; créer des abstractions spécifiques.** Un concept unique ("scénario") couvrant deux patterns différents (*simulation* et *préprod*) invite à la confusion. Les nommer distinctement et leur donner des UI, règles et durées de vie différentes est plus clair et produit un code plus simple.
- **Concevoir pour les cas concrets avant de concevoir pour le cas général.** L'architecture overlay était techniquement élégante mais résolvait des problèmes que l'utilisateur n'avait pas. Les cas d'usage concrets sont la seule source honnête de besoins.
- **La séparation planning/exécution est un pattern ancien et éprouvé.** Ne pas le réinventer ; le nommer et emprunter les disciplines connues (rituel de commit, flux unidirectionnels, réconciliation limitée au feedback d'exécution).
- **Le modèle mental de l'utilisateur n'est pas le modèle de données.** Quand l'utilisateur dit "je modifie le job" et entend "je change l'affectation machine", la mutation de données est

localisée sur une ligne de `TaskAssignment`, pas sur l'entité Job. Le nom ("job") et la donnée du même nom ne sont pas toujours la même chose.

- **L'ambient awareness compte quand la confusion de contexte a des conséquences.** Dans les systèmes multi-environnements, le coût d'agir dans le mauvais environnement détermine à quel point l'indicateur de contexte doit être bruyant. Libre d'être subtil si les conséquences sont faibles ; obligatoire d'être bruyant si elles sont lourdes.
- **Préférer les copies complètes aux overlays quand les use cases n'exigent pas de synchronisation.** Les overlays sont puissants mais portent leur complexité (logique de projection, opérations structurelles, sémantique des conflits). Les copies complètes sont plus faciles à raisonner et suffisantes quand les environnements sont peu nombreux, single-user, et courts ou clairement stagés.

4. Inspirations citées

Issues de la discussion, regroupées par catégorie :

APS industriels : SAP APO (planning versions), Dassault Quintiq, Siemens Opcenter — tous ont des concepts de scénarios/versions ; tous imposent la séparation planning-vs-execution.

Vernionnement logiciel : Git (branches, commits, diff, tag — pertinents ; merge 3-way, rebase linéaire — pas pertinents pour les graphes de planification).

Systèmes overlay : Kubernetes Kustomize (base + overlays + patches), Docker layers (add/remove/modify à travers les couches), Figma component instances (master + overrides par instance), vues Airtable (table + filtres/tris par vue), vues de base Notion.

Patterns UX pour le switch de contexte : Figma branching (non-devs utilisant branch/review/merge), Linear cycles, Excel Scenario Manager, versions nommées Google Docs, picker repo/org GitHub (le pattern *le-nom-du-contexte-est-le-switcher*), switcher de workspace Slack.

Discipline environnementale (ambient awareness) : bande rouge macOS quand l'écran est partagé, bandeaux de sandbox Jira, coloration Chrome incognito.

5. Ce qui a été exploré et rejeté

Pour exhaustivité, les décisions *non* prises :

Choix rejeté	Raison
Système multi-scénarios nommés avec couleurs	Les use cases réels ont besoin de deux environnements fixes + bulles éphémères, pas d'une forêt de branches
Modèle overlay / delta (réalité partagée + overlay par scénario)	Élégant mais pas nécessaire ; pas de pression de staleness et pas de collaboration multi-utilisateur sur même environnement
Rôle d'approbateur distinct du planificateur	Le chef d'atelier est l'approbateur par définition de son rôle
Co-édition des scénarios par plusieurs planificateurs	Un seul planificateur par société ; la collaboration passe par le rituel de promotion, pas par l'édition simultanée
UX de staleness / merge-conflict	Avec le flux continu et une seule préprod, le problème n'apparaît pas
UI de comparaison côte-à-côte entre scénarios	Explicitement pas nécessaire — la comparaison se fait dans deux onglets séparés si besoin
Dashboards KPI par scénario	Pas nécessaire vu qu'il n'y a pas de flow de comparaison
Hot-patch direct de la prod en urgence	La cadence de promotion fréquente rend le chemin <i>re-build en préprod</i> suffisamment rapide (Option A choisie plutôt que l'Option B)
Règles de rétention / purge des archives	Les archives sont conservées indéfiniment ; curation manuelle seulement si le stockage devient un enjeu

6. État final du modèle mental

Après le parcours complet :

- L'app actuelle = la future **préprod**. Aucun changement de comportement pour le chef d'atelier.
- Une nouvelle **vue prod** est ajoutée : rendu en lecture seule du plan engagé, avec contrôles de feedback d'avancement.
- Une **action de promotion** est introduite : le seul moment où le plan traverse la frontière préprod → prod.

- Un nouveau **point d'entrée simulation** est ajouté pour les ADV : ouvre un nouvel onglet navigateur avec un fork isolé de la préprod.
- Deux workers background font la colle : pousse réservoir → préprod pour les nouveaux jobs, sync prod → préprod pour la completion.

La feature est beaucoup plus petite que ce que suggérait la question initiale. C'est précisément le point.